

Serverless Data Pipeline Using AWS

Objective

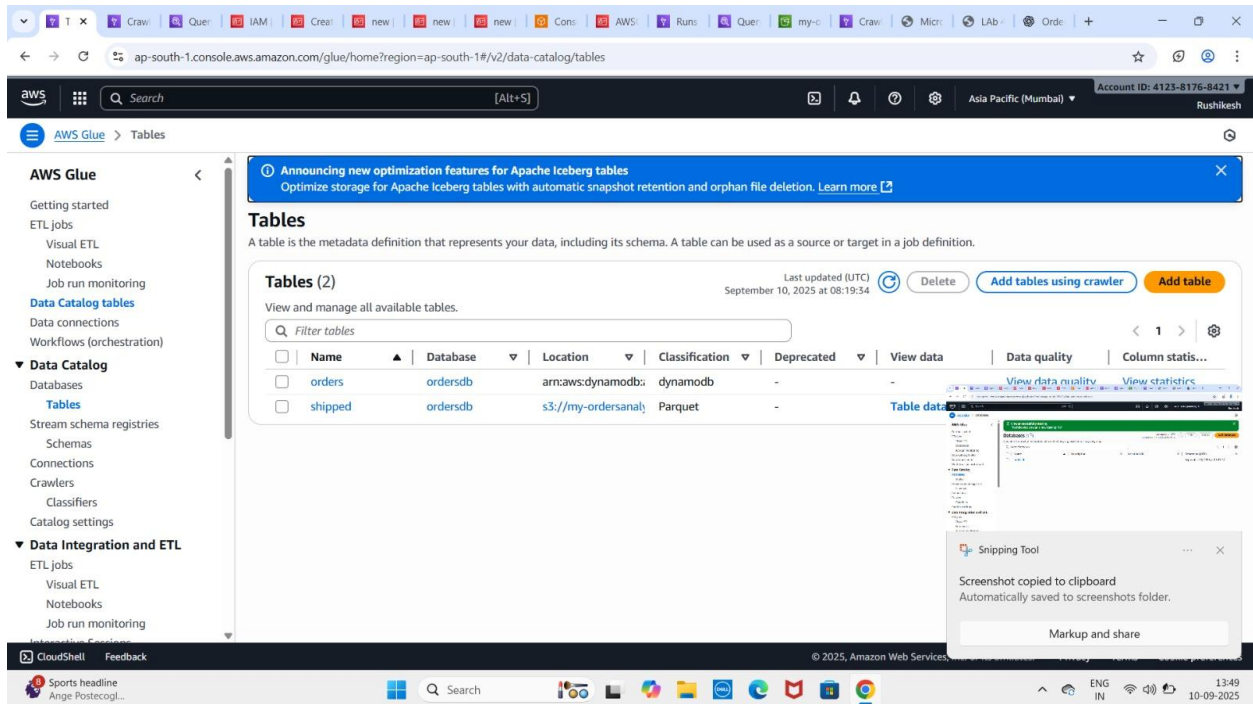
Build a serverless ETL pipeline using **DynamoDB → AWS Glue → S3 → Athena**. You will:

- Create a DynamoDB Orders table.
- Insert sample order data.
- Use **two Glue Crawlers** (DynamoDB + S3).
- Configure IAM roles and **two different Groups** (restricted vs full access).
- Build a Glue ETL job to filter "Shipped" orders into S3.
- Query processed data with Athena.

Step 1: Create the Orders DynamoDB Table

1. Sign in to **AWS Console** → Services → **DynamoDB**.
2. Click **Create Table**.
3. Enter table details:
 - Table name: **Orders**
 - Primary key:
 - Partition key: OrderID (String)
 - Sort key: OrderDate (String)
4. Under **Capacity mode** → select **On-demand**.
5. Click **Create Table**.

Screenshot:



Step 2: Insert Sample Data

- Your table has keys: OrderID (PK) and OrderDate (SK).
- Use ISO-8601 format for dates (YYYY-MM-DDTHH:MM:SSZ).
- Insert 15 sample orders (JSON format).

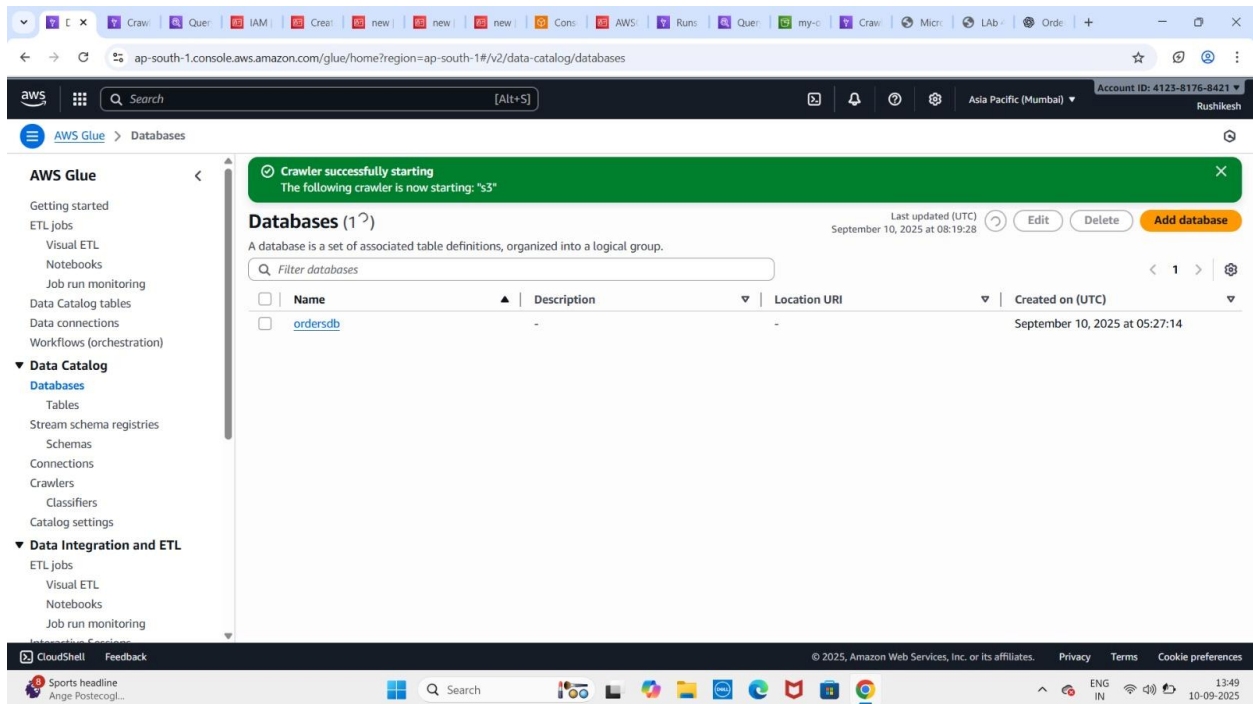
```
{  
  "OrderID": { "S": "O1001" },  
  "OrderDate": { "S": "2025-09-01T09:10:00Z" },  
  "Customer": { "S": "Alice" },  
  "Amount": { "N": "250" },  
  "Status": { "S": "Shipped" }  
}
```

Step 3: Create Glue Crawlers

3.1 Create Glue Database

1. Go to **AWS Glue** → Databases.
2. Click **Add Database**.
3. Name: **ordersdb**.
4. Click **Create**.

 **Screenshot:**

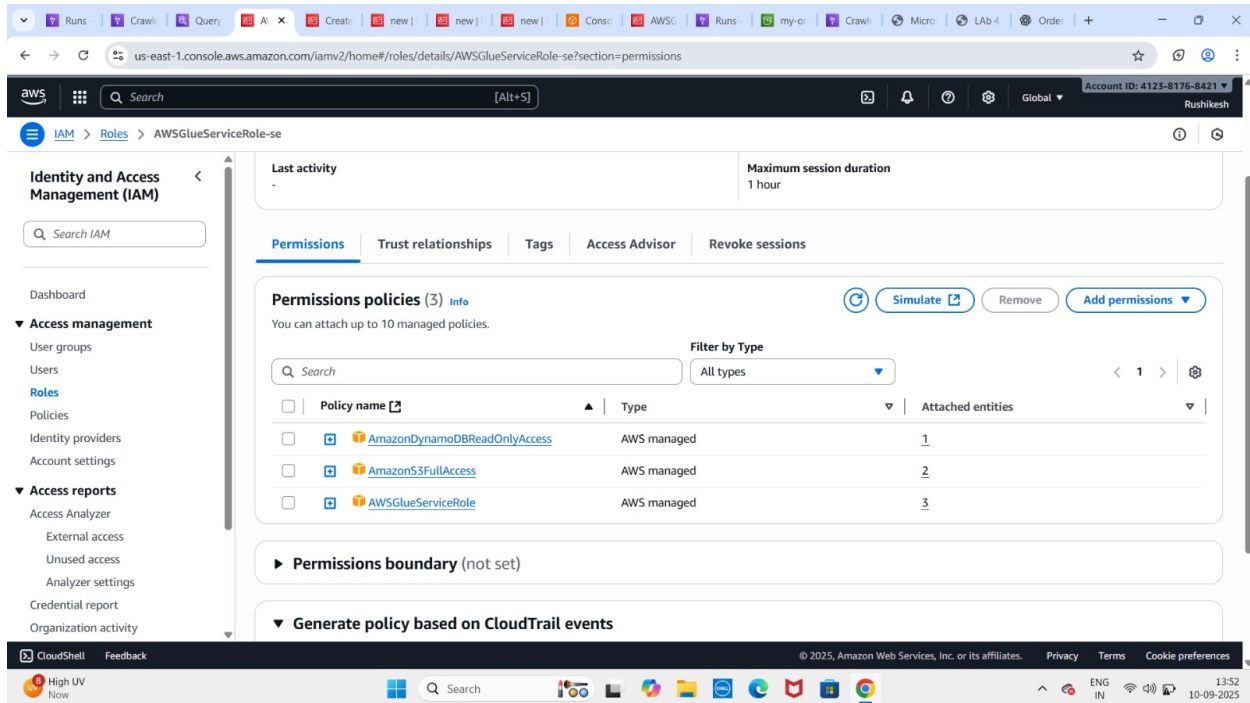


3.2 Create IAM Roles (Two Variants)

◆ Group 1 (Minimal Attached Policies Only)

- Attached Policies:
 - AmazonDynamoDBReadOnlyAccess
 - AmazonS3ReadOnlyAccess
 - AWSGlueServiceRole

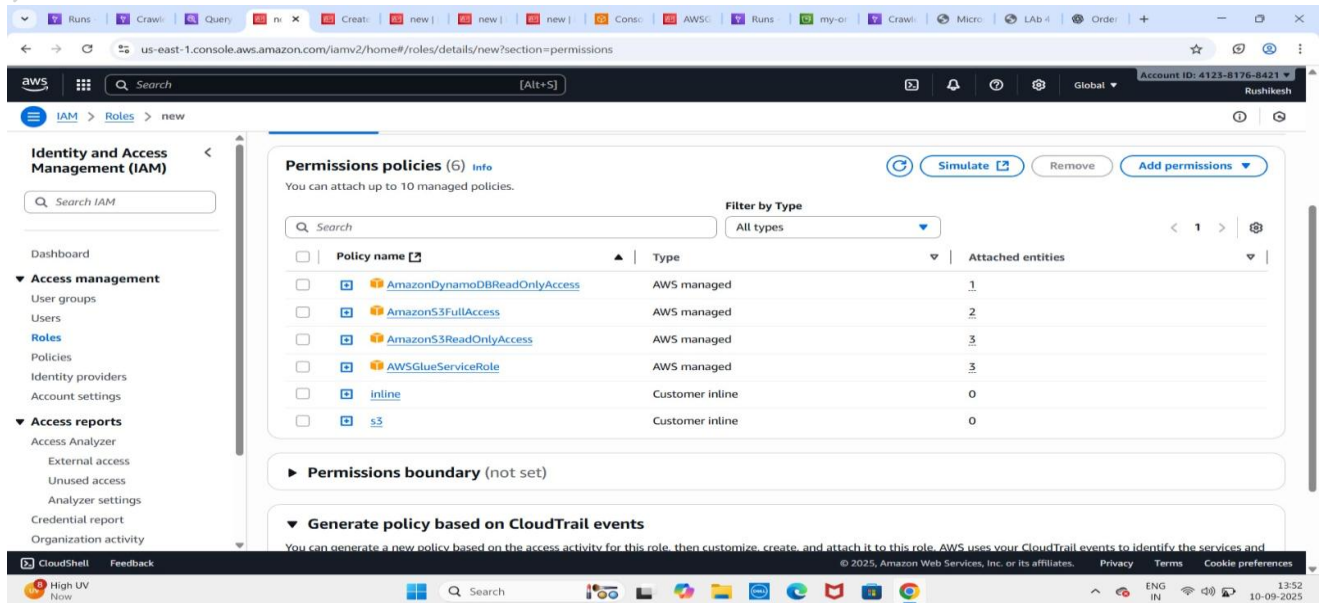
 **Screenshot:**



◆ Group 2 (All Policies Inline + Attached)

- Inline S3 Full Access Policy (example):
- Attached Policies:
 - AmazonDynamoDBReadOnlyAccess
 - AmazonS3FullAccess
 - AWSGlueServiceRole
 - DynamoDB read: DescribeTable, Scan, GetItem, Query for the Orders table ARN.
 - S3 access to your bucket: GetObject, PutObject, ListBucket for arn:aws:s3:::my-orders-analy-cs01
- Trust Policy remains same but **includes S3 explicitly if required**.

Screenshot:

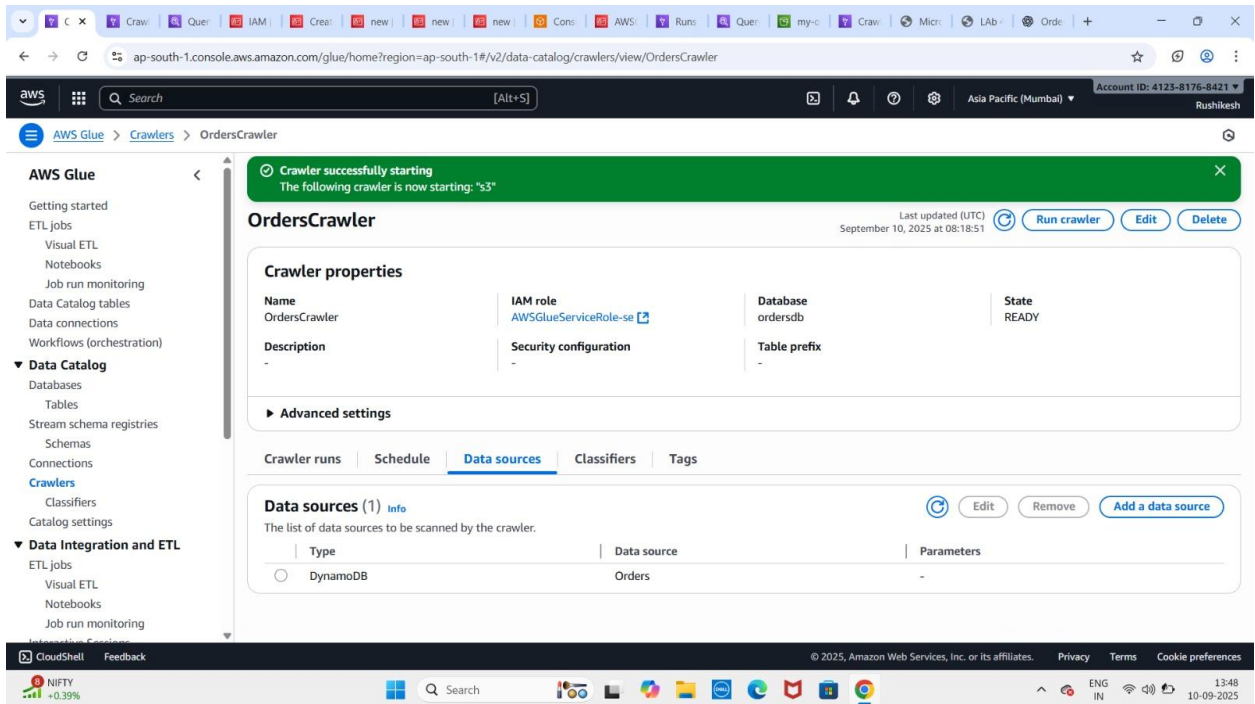


3.3 Create Crawlers

◆ **Crawler 1: DynamoDB → Glue**

1. Go to **Glue** → **Crawlers** → **Create Crawler**.
2. Name: **OrdersDynamoCrawler**.
3. Data Source: **DynamoDB** → select **Orders** table.
4. IAM Role: Choose **G1 IAM Role**.
5. Target Database: **ordersdb**.
6. Run on-demand.

Screenshot:

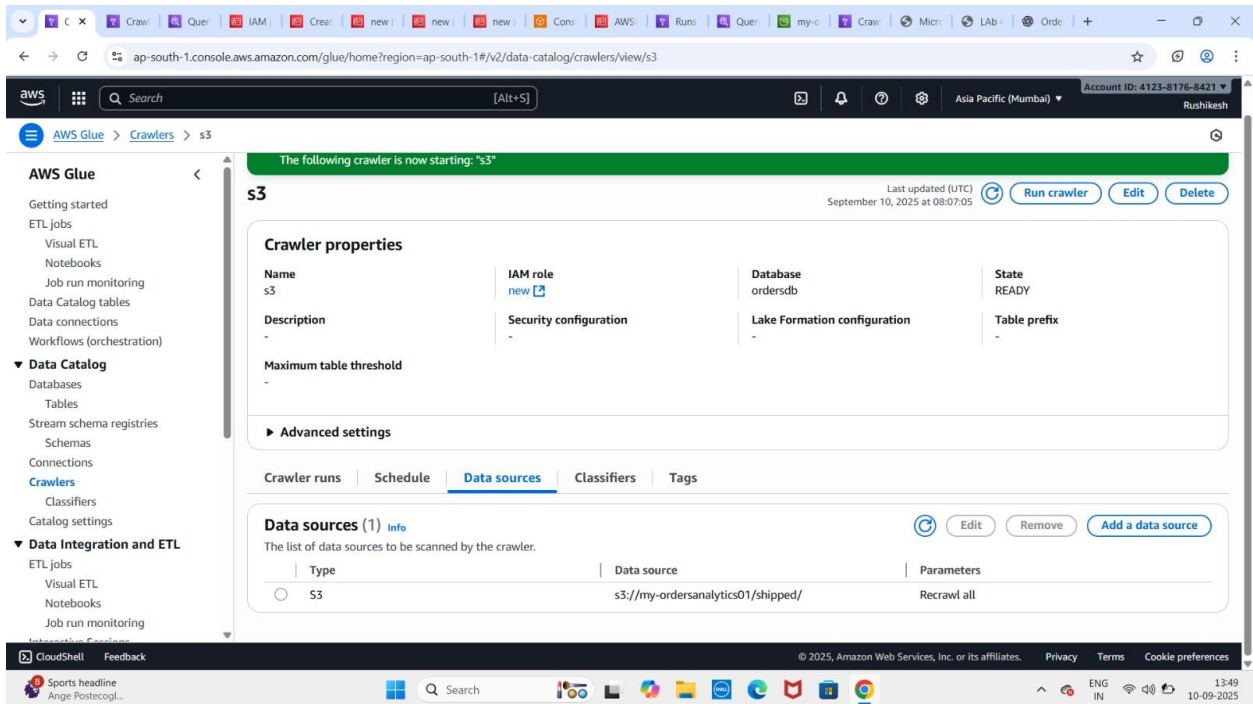


The screenshot displays the AWS Glue console interface. At the top, a green banner indicates "Crawler successfully starting" with the message "The following crawler is now starting: 's3'". Below this, the "OrdersCrawler" configuration is shown. The crawler properties include: Name: OrdersCrawler, IAM role: AWSGlueServiceRole-se, Database: ordersdb, and State: READY. The description is empty, and the security configuration and table prefix are also empty. The "Advanced settings" section is collapsed. The "Data sources" tab is selected, showing a table with one data source: Type: DynamoDB, Data source: Orders, and Parameters: -. The left sidebar shows the navigation menu with categories like "Getting started", "Data Catalog", and "Data Integration and ETL". The bottom of the screen shows the Windows taskbar with various application icons and the system clock.

◆ Crawler 2: S3 → Glue

1. Go to **Glue → Crawlers → Create Crawler**.
2. Name: **OrdersS3Crawler**.
3. Data Source: **S3** → bucket path: **s3://my-orders-analytics01/shipped/**.
4. IAM Role: Choose **G2 IAM Role**.
5. Target Database: **ordersdb**.
6. Run on-demand.

Screenshot:



Step 4: Create AWS Glue ETL Job (Transformation)

 Glue ETL (Spark) will **filter only shipped orders** and write to **S3 in Parquet format**.

1. Go to **AWS Glue** → **Jobs** → **Add job**

- **Name:** OrdersETLJob
- **IAM Role:** Use same role with DynamoDB + S3 access.(G2)
- **Source:** From Catalog → ordersdb.orders
- **Target:** S3 → my-orders-analytics (create bucket before).
- **Format:** Parquet.

2. script:

- import sys
- import re
- from awsglue.transforms import *
- from awsglue.utils import getResolvedOptions
- from pyspark.context import SparkContext
- from awsglue.context import GlueContext
- from awsglue.job import Job
- from awsglue.dynamicframe import DynamicFrame
- import pyspark.sql.functions as F

```

12. # --- JOB ARGS ---
13. args = getResolvedOptions(sys.argv, ['JOB_NAME'])
14. sc = SparkContext()
15. glueContext = GlueContext(sc)
16. spark = glueContext.spark_session
17. job = Job(glueContext)
18. job.init(args['JOB_NAME'], args)
19. # --- CONFIG ---
20. DATABASE = "ordersdb"
21. TABLE_NAME = "orders"
22. OUTPUT_PATH = "s3://my-orders-analysis01/shipped/"
23. # --- READ from Glue Data Catalog (DynamoDB table) ---
24. datasource_dyf = glueContext.create_dynamic_frame.from_catalog(
25.     database=DATABASE,
26.     table_name=TABLE_NAME,
27.     transformation_ctx="datasource_dyf"
28. )
29. # Convert to Spark DataFrame
30. df = datasource_dyf.toDF()
31. # --- FIX Amount column ---
32. # DynamoDB "choice" type: struct with {double, long}
33. df = df.withColumn(
34.     "Amount",
35.     F.when(F.col("Amount.double").isNotNull(), F.col("Amount.double"))
36.     .when(F.col("Amount.long").isNotNull(), F.col("Amount.long").cast("double"))
37.     .otherwise(F.lit(None))
38. )
39. # --- FILTER shipped orders (case-insensitive) ---
40. f
41. filtered_df = df.filter(F.lower(F.col("Status")) == "shipped")
42. # Convert back to DynamicFrame
43. f
44. filtered_dyf = DynamicFrame.fromDF(filtered_df, glueContext, "filtered_dyf")
45. # --- WRITE to S3 ---
46. glueContext.write_dynamic_frame.from_options(
47.     frame=filtered_dyf,
48.     connection_type="s3",
49.     connection_options={"path": OUTPUT_PATH, "partitionKeys": []},
50.     format="parquet",
51.     format_options={"compression": "snappy"},
52.     transformation_ctx="s3_sink"
53. )
54. job.commit()

```


}

📌 Screenshot:

The screenshot displays the AWS Glue Studio console interface. The left sidebar contains navigation links for AWS Glue, including 'Getting started', 'ETL jobs', 'Data Catalog', and 'Data Integration and ETL'. The main content area is titled 'AWS Glue Studio' and features a 'Create job' section with three options: 'Visual ETL' (authoring in a visual interface), 'Notebook' (authoring using an interactive code notebook), and 'Script editor' (authoring code with a script editor). Below this is an 'Example jobs' section with a 'Create example job' button. The 'Your jobs (1)' section shows a table of existing jobs.

Job name	Type	Created by	Last modified	AWS Glue version
OrdersToParquet	Glue ETL	Script	10/09/2025, 12:50:55	5.0

The bottom of the screenshot shows the Windows taskbar with various application icons and the system clock indicating 13:49 on 10-09-2025.

ap-south-1.console.aws.amazon.com/gluestudio/home?region=ap-south-1#/editor/job/OrdersToParquet/script

Search [Alt+S] Asia Pacific (Mumbai) Account ID: 4123-8176-8421 Rushikesh

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Data Catalog
 - Databases
 - Tables
 - Stream schema registries
 - Schemas
 - Connections
 - Crawlers
 - Classifiers
 - Catalog settings
- Data Integration and ETL
 - ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring

OrdersToParquet

Last modified on 10/09/2025, 12:50:55 [Actions](#) [Save](#) [Run](#)

[Script](#) [Job details](#) [Runs](#) [Data quality](#) [Schedules](#) [Version Control](#)

Script Info

```
1 import sys
2 from aws glue.transforms import *
3 from aws glue.utils import getResolvedOptions
4 from pyspark.context import SparkContext
5 from aws glue.context import GlueContext
6 from aws glue.job import Job
7 from aws glue.dynamicframe import DynamicFrame
8 import pyspark.sql.functions as F
9
10 # --- JOB ARGS ---
11 args = getResolvedOptions(sys.argv, ['JOB_NAME'])
12
13 sc = SparkContext()
14 glueContext = GlueContext(sc)
15 spark = glueContext.spark_session
16
17 job = Job(glueContext)
18 job.init(args['JOB_NAME'], args)
```

Python Ln 1, Col 1 Errors: 0 Warnings: 0

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13:49 10-09-2025

ap-south-1.console.aws.amazon.com/gluestudio/home?region=ap-south-1#/editor/job/OrdersToParquet/runs

Search [Alt+S] Asia Pacific (Mumbai) Account ID: 4123-8176-8421 Rushikesh

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Data Catalog
 - Databases
 - Tables
 - Stream schema registries
 - Schemas
 - Connections
 - Crawlers
 - Classifiers
 - Catalog settings
- Data Integration and ETL
 - ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring

OrdersToParquet

Last modified on 10/09/2025, 12:50:55 [Actions](#) [Save](#) [Run](#)

[Script](#) [Job details](#) [Runs](#) [Data quality](#) [Schedules](#) [Version Control](#)

Job runs (1/7) Info

Last updated (UTC) September 10, 2025 at 08:20:04 [View details](#) [Stop job run](#) [Troubleshoot with AI](#) [Table View](#) [Card View](#)

Filter job runs by property

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (...)	Worker type	Glue
Succeeded	0	09/10/2025 12:51:10	09/10/2025 12:52:30	1 m 8 s	10 DPUs	G.1X	5.0
Succeeded	0	09/10/2025 12:41:04	09/10/2025 12:42:21	1 m 9 s	10 DPUs	G.1X	5.0

Run details

Input arguments (9)	Logs	Run insights	Metrics	Troubleshooting analysis - p
Job name OrdersToParquet	Start time (Local) 09/10/2025 12:51:10	Glue version 5.0	Worker type G.1X	
Id jr_0606e51fc2f98f0e427dd534618b4f37f69	End time (Local) 09/10/2025 12:52:30	Max capacity 10 DPUs	Execution class	
Run status Succeeded	Start-up time 11 seconds			
Retry attempt number	Execution time			

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Markup and share

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13:50 10-09-2025

Step 5: Query with Athena (Analytics)

✦ Athena lets you run **SQL directly on S3 data**.

1. Go to **Athena Console**.
2. Set **Query result location**: e.g., s3://athena-query-results-bucket/.
3. In **Database**, select ordersdb.

Example Queries:

```
SELECT Customer, SUM(Amount) AS TotalSpent
FROM orders_shipped
WHERE lower(Status) = 'shipped'
GROUP BY Customer
ORDER BY TotalSpent DESC
LIMIT 20;
```

🎯 Final Result

You built a **serverless pipeline** that:

- Stores e-commerce data in **DynamoDB**
- Uses **Glue Crawler** for schema discovery
- Runs **Glue ETL** to filter & store clean data in **S3 (Data Lake)**
- Uses **Athena** for SQL analytics